



Convert TensorFlow, Keras, Tensorflow.js and Tflite models to ONNX

Apache-2.0 license

☆ 2.4k stars

🍴 436 forks

👁 53 watching

🔗 Branches

🏠 Activity

📄 Custom properties

🏷 Tags

Public repository

🔗

🔗 28 Branches

🏷 33 Tags

🔗

🏷

🔍 Go to file

t

Go to file

+

Add file ▾

Code

...

 **xadupre**

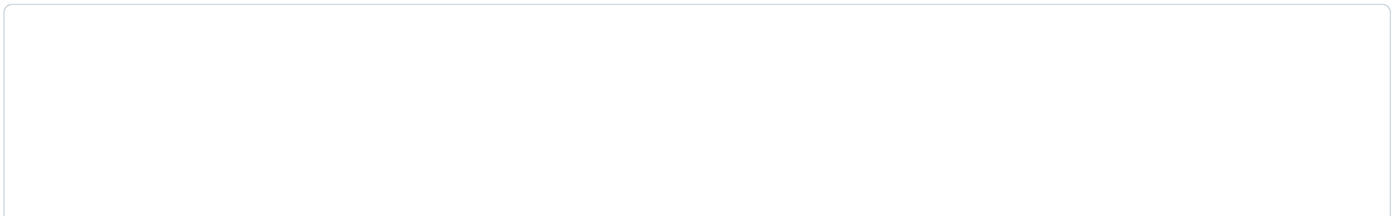
Merge pull request [#1960](#) from hwangdeyu/optimize_shape_infer

✓

3dd7729 · 2 months ago

🕒

📁 .github	switch to v4	2 months ago
📁 examples	Update end2end_tfkeras.py (#2361)	4 months ago
📁 tests	fix one test	2 months ago
📁 tf2onnx	feat: add support for ExtractImagePa...	4 months ago
📁 tools	Merge branch 'main' into optimize_s...	2 months ago
📁 tutorials	Some minor fixes for keras-resnet50...	2 years ago
📄 .gitignore	switch to v4	2 months ago
📄 CONTRIBUTING.md	Update CONTRIBUTING.md CLA to ...	3 years ago
📄 LICENSE	remove leftover MIT licenses (#1272)	5 years ago
📄 README.md	Update README.md (#2381)	3 months ago
📄 Troubleshooting.md	Minor doc tf 2.8 change (#1934)	3 years ago
📄 VERSION_NUMBER	Prepare updates for release 1.16.1. (#...	last year
📄 build.bat	add ALv2 licenses (#1271)	5 years ago
📄 build.sh	add ALv2 licenses (#1271)	5 years ago
📄 setup.cfg	Add windows keras2onnx application...	4 years ago
📄 setup.py	Correct the required version of proto...	2 years ago
📄 support_status.md	Support tf ResizeArea op. (#2241)	2 years ago



tf2onnx - Convert TensorFlow, Keras, Tensorflow.js and Tflite models to ONNX.

tf2onnx converts TensorFlow (tf-1.x or tf-2.x), keras, tensorflow.js and tflite models to ONNX via command line or python api.

Note: tensorflow.js support was just added. While we tested it with many tfjs models from tfhub, it should be considered experimental.

TensorFlow has many more ops than ONNX and occasionally mapping a model to ONNX creates issues.

You find a list of supported TensorFlow ops and their mapping to ONNX [here](#).

The common issues we run into we try to document here [Troubleshooting Guide](#).

Build Type	OS	Python	TensorFlow	ONNX opset	Status
Unit Test - Basic	Linux, Windows	3.7-3.10	1.15, 2.9-2.15	14-18	 Build Status
Unit Test - Full	Linux, Windows	3.7-3.10	1.15, 2.9-2.15	14-18	 Build Status

Supported Versions

ONNX

tf2onnx will use the ONNX version installed on your system and installs the latest ONNX version if none is found.

We support and test ONNX opset-14 to opset-18. opset-6 to opset-13 should work but we don't test them. By default we use opset-15 for the resulting ONNX graph.

If you want the graph to be generated with a specific opset, use `--opset` in the command line, for example `--opset 15`.

TensorFlow

We support tf-1.x graphs and tf-2.x. To keep our test matrix manageable we test tf2onnx running on top of tf-1.15 or better.

When running under tf-2.x tf2onnx will use the tensorflow V2 controlflow.

You can install tf2onnx on top of tf-1.x or tf-2.x.

Python

We support Python 3.7-3.10.

Prerequisites

Install TensorFlow

If you don't have TensorFlow installed already, install the desired TensorFlow build, for example:

```
pip install tensorflow
```

(Optional) Install runtime

If you want to run tests, install a runtime that can run ONNX models. For example:

ONNX Runtime (available for Linux, Windows, and Mac):

```
pip install onnxruntime
```

Installation

Install from pypi

```
pip install -U tf2onnx
```

Install latest from github

```
pip install git+https://github.com/onnx/tensorflow-onnx
```

Build and install latest from source (for development)

```
git clone https://github.com/onnx/tensorflow-onnx
```

Once dependencies are installed, from the tensorflow-onnx folder call:

```
python setup.py install
```

or

```
python setup.py develop
```

tensorflow-onnx requires onnx-1.9 or better and will install/upgrade onnx if needed.

To create a wheel for distribution:

```
python setup.py bdist_wheel
```

Getting started

To get started with `tensorflow-onnx`, run the `tf2onnx.convert` command, providing:

- the path to your TensorFlow model (where the model is in `saved model` format)
- a name for the ONNX output file:

```
python -m tf2onnx.convert --saved-model tensorflow-model-path --output model.onnx
```

The above command uses a default of `15` for the ONNX opset. If you need a newer opset, or want to limit your model to use an older opset then you can provide the `--opset` argument to the command. If you are unsure about which opset to use, refer to the [ONNX operator documentation](#).

```
python -m tf2onnx.convert --saved-model tensorflow-model-path --opset 18 --output model.onnx
```

If your TensorFlow model is in a format other than `saved model`, then you need to provide the inputs and outputs of the model graph.

For `checkpoint` format:

```
python -m tf2onnx.convert --checkpoint tensorflow-model-meta-file-path --output model.onnx --inputs input0:0,input1:0 --outputs output0:0
```

For `graphdef` format:

```
python -m tf2onnx.convert --graphdef tensorflow-model-graphdef-file --output model.onnx --inputs input0:0,input1:0 --outputs output0:0
```

If your model is in `checkpoint` or `graphdef` format and you do not know the input and output nodes of the model, you can use the [summarize_graph](#) TensorFlow utility. The `summarize_graph` tool does need to be downloaded and built from source. If you have the option of going to your model provider and obtaining the model in `saved_model` format, then we recommend doing so.

You find an end-to-end tutorial for `ssd-mobilenet` [here](#)

We recently added support for `tflite`. You convert `tflite` models via command line, for example:

```
python -m tf2onnx.convert --opset 16 --tflite tflite-file --output model.onnx
```

CLI reference

```
python -m tf2onnx.convert
--saved-model SOURCE_SAVED_MODEL_PATH |
--checkpoint SOURCE_CHECKPOINT_METAFILE_PATH |
--tflite TFLITE_MODEL_PATH |
--tfjs TFJS_MODEL_PATH |
--input | --graphdef SOURCE_GRAPHDEF_PB
--output TARGET_ONNX_MODEL
[--inputs GRAPH_INPUTS]
[--outputs GRAPH_OUTPUTS]
[--inputs-as-nchw inputs_provided_as_nchw]
[--outputs-as-nchw outputs_provided_as_nchw]
[--opset OPSET]
[--dequantize]
[--tag TAG]
[--signature_def SIGNATURE_DEF]
[--concrete_function CONCRETE_FUNCTION]
[--target TARGET]
[--extra_opset list-of-extra-opset]
[--custom-ops list-of-custom-ops]
[--load_op_libraries tensorflow_library_path]
[--large_model]
[--continue_on_error]
[--verbose]
[--output_frozen_graph]
```



Parameters

--saved-model

TensorFlow model as `saved_model`. We expect the path to the `saved_model` directory.

--checkpoint

TensorFlow model as `checkpoint`. We expect the path to the `.meta` file.

--input or --graphdef

TensorFlow model as `graphdef` file.

--tfjs

Convert a `tensorflow.js` model by providing a path to the `.tfjs` file. Inputs/outputs do not need to be specified.

--tflite

Convert a `tflite` model by providing a path to the `.tflite` file. Inputs/outputs do not need to be specified.

--output

The target onnx file path.

--inputs, --outputs

TensorFlow model's input/output names, which can be found with [summarize graph tool](#). Those names typically end with `:0`, for example `--inputs input0:0,input1:0`. Inputs and outputs are *not* needed for models in saved-model format. Some models specify placeholders with unknown ranks and dims which can not be mapped to onnx. In those cases one can add the shape after the input name inside `[]`, for example `--inputs X:0[1,28,28,3]`. Use `-1` to indicate unknown dimensions.

--inputs-as-nchw

By default we preserve the image format of inputs (`nchw` or `nhwc`) as given in the TensorFlow model. If your hosts (for example windows) native format `nchw` and the model is written for `nhwc`, `--inputs-as-nchw` tensorflow-onnx will transpose the input. Doing so is convenient for the application and the converter in many cases can optimize the transpose away. For example `--inputs input0:0,input1:0 --inputs-as-nchw input0:0` assumes that images are passed into `input0:0` as `nchw` while the TensorFlow model given uses `nhwc`.

--outputs-as-nchw

Similar usage with `--inputs-as-nchw`. By default we preserve the format of outputs (`nchw` or `nhwc`) as shown in the TensorFlow model. If your hosts native format `nchw` and the model is written for `nhwc`, `--outputs-as-nchw` tensorflow-onnx will transpose the output and optimize the transpose away. For example `--outputs output0:0,output1:0 --outputs-as-nchw output0:0` will change the `output0:0` as `nchw` while the TensorFlow model given uses `nhwc`.

--ignore_default, --use_default

ONNX requires default values for graph inputs to be constant, while Tensorflow's PlaceholderWithDefault op accepts computed defaults. To convert such models, pass a comma-separated list of node names to the `ignore_default` and/or `use_default` flags. PlaceholderWithDefault nodes with matching names will be replaced with Placeholder or Identity ops, respectively.

--opset

By default we use the opset 15 to generate the graph. By specifying `--opset` the user can override the default to generate a graph with the desired opset. For example `--opset 17` would create a onnx graph that uses only ops available in opset 17. Because older opsets have in most cases fewer ops, some models might not convert on an older opset.

--dequantize

(This is experimental, only supported for tflite)

Produces a float32 model from a quantized tflite model. Detects ReLU and ReLU6 ops from quantization bounds.

--tag

Only valid with parameter `--saved_model`. Specifies the tag in the saved_model to be used. Typical value is 'serve'.

--signature_def

Only valid with parameter `--saved_model`. Specifies which signature to use within the specified `--tag` value. Typical value is 'serving_default'.

--concrete_function

(This is experimental, valid only for TF2.x models)

Only valid with parameter `--saved_model` . If a model contains a list of concrete functions, under the function name `__call__` (as can be viewed using the command `saved_model_cli show --all`), this parameter is a 0-based integer specifying which function in that list should be converted. This parameter takes priority over `--signature_def` , which will be ignored.

--target

Some models require special handling to run on some runtimes. In particular, the model may use unsupported data types. Workarounds are activated with `--target TARGET` . Currently supported values are listed on this [wiki](#). If your model will be run on Windows ML, you should specify the appropriate target value.

--extra_opset

If you want to convert a TF model using an existing custom op, this can specify the corresponding domain and version. The format is a comma-separated map of domain and version, for example: `ai.onnx.contrib:1` .

--custom-ops

If a model contains ops not recognized by onnx runtime, you can tag these ops with a custom op domain so that the runtime can still open the model. The format is a comma-separated map of tf op names to domains in the format `OpName:domain`. If only an op name is provided (no colon), the default domain of `ai.onnx.converters.tensorflow` will be used.

--load_op_libraries

Load the comma-separated list of tensorflow plugin/op libraries before conversion.

--large_model

(Can be used only for TF2.x models)

Only valid with parameter `--saved_model` . When set, creates a zip file containing the ONNX protobuf model and large tensor values stored externally. This allows for converting models whose size exceeds the 2 GB.

--continue_on_error

Continue to run conversion on error, ignore graph cycles so it can report all missing ops and errors.

--verbose

Verbose detailed output for diagnostic purposes.

--output_frozen_graph

Save the frozen and optimized tensorflow graph to a file for debug.

Tool to get Graph Inputs & Outputs

To find the inputs and outputs for the TensorFlow graph the model developer will know or you can consult TensorFlow's [summarize_graph](#) tool, for example:

```
summarize_graph --in_graph=tests/models/fc-layers/frozen.pb
```



Testing

There are 2 types of tests.

Unit test

```
python setup.py test
```



Validate pre-trained TensorFlow models

```
python tests/run_pretrained_models.py
usage: run_pretrained_models.py [-h] [--cache CACHE] [--tests TESTS] [--backend BACKEND]
[--verbose] [--debug] [--config yaml-config]
```



optional arguments:

-h, --help	show this help message and exit
--cache CACHE	pre-trained models cache dir
--tests TESTS	tests to run
--backend BACKEND	backend to use
--config	yaml config file
--verbose	verbose output, option is additive
--opset OPSET	target opset to use
--perf csv-file	capture performance numbers for tensorflow and onnx runtime
--debug	dump generated graph with shape info

`run_pretrained_models.py` will run the TensorFlow model, captures the TensorFlow output and runs the same test against the specified ONNX backend after converting the model.

If the option `--perf csv-file` is specified, we'll capture the timing for inference of tensorflow and onnx runtime and write the result into the given csv file.

You call it for example with:

```
python tests/run_pretrained_models.py --backend onnxruntime --config
tests/run_pretrained_models.yaml --perf perf.csv
```



Tool to save pre-trained model

We provide an [utility](#) to save pre-trained model along with its config. Put `save_pretrained_model(sess, outputs, feed_inputs, save_dir, model_name)` in your last testing epoch and the pre-trained model and config will be saved under `save_dir/to_onnx`. Please refer to the example in [tools/save_pretrained_model.py](#) for more information. Note the minimum required Tensorflow version is r1.6.

Python API Reference

With tf2onnx-1.8.4 we updated our API. Our old API still works - you find the documentation [here](#).

from_keras (tf-2.0 and newer)

```
import tf2onnx

model_proto, external_tensor_storage = tf2onnx.convert.from_keras(model,
    input_signature=None, opset=None, custom_ops=None,
    custom_op_handlers=None, custom_rewriter=None,
    inputs_as_nchw=None, outputs_as_nchw=None, extra_opset=None,
    shape_override=None, target=None, large_model=False, output_path=None)
```



Args:

`model`: the tf.keras model we want to convert
`input_signature`: a tf.TensorSpec or a numpy array defining the shape/dtype of the

input

`opset`: the opset to be used for the ONNX model, default is the latest
`custom_ops`: if a model contains ops not recognized by onnx runtime, you can tag these ops with a custom op domain so that the runtime can still open the model. Type is a dictionary `{op name: domain}`.

target: list of workarounds applied to help certain platforms
custom_op_handlers: dictionary of custom ops handlers
custom_rewriter: list of custom graph rewriters
extra_opset: list of extra opset's, for example the opset's used by custom ops
shape_override: dict with inputs that override the shapes given by tensorflow
inputs_as_nchw: transpose inputs in list from nhwc to nchw
outputs_as_nchw: transpose outputs in list from nhwc to nchw
large_model: use the ONNX external tensor storage format
output_path: save model to output_path

Returns:

An ONNX model_proto and an external_tensor_storage dict.

See [tutorials/keras-resnet50.ipynb](https://tutorials.keras-resnet50.ipynb) for an end to end example.

from_function (tf-2.0 and newer)

```
import tf2onnx
```



```
model_proto, external_tensor_storage = tf2onnx.convert.from_function(function,  
    input_signature=None, opset=None, custom_ops=None,  
    custom_op_handlers=None, custom_rewriter=None, inputs_as_nchw=None,  
    outputs_as_nchw=None, extra_opset=None, shape_override=None,  
    target=None, large_model=False, output_path=None)
```

Args:

function: the tf.function we want to convert

input_signature: a tf.TensorSpec or a numpy array defining the shape/dtype of the

input

opset: the opset to be used for the ONNX model, default is the latest

custom_ops: if a model contains ops not recognized by onnx runtime,
you can tag these ops with a custom op domain so that the
runtime can still open the model. Type is a dictionary `{op name: domain}`.

target: list of workarounds applied to help certain platforms

custom_op_handlers: dictionary of custom ops handlers

custom_rewriter: list of custom graph rewriters

extra_opset: list of extra opset's, for example the opset's used by custom ops

shape_override: dict with inputs that override the shapes given by tensorflow

inputs_as_nchw: transpose inputs in list from nhwc to nchw

outputs_as_nchw: transpose outputs in list from nhwc to nchw

large_model: use the ONNX external tensor storage format

output_path: save model to output_path

Returns:

An ONNX model_proto and an external_tensor_storage dict.

from_graph_def

```
import tf2onnx
```



```
model_proto, external_tensor_storage = tf2onnx.convert.from_graph_def(graph_def,  
    name=None, input_names=None, output_names=None, opset=None,  
    custom_ops=None, custom_op_handlers=None, custom_rewriter=None,  
    inputs_as_nchw=None, outputs_as_nchw=None, extra_opset=None,  
    shape_override=None, target=None, large_model=False,  
    output_path=None)
```

Args:

graph_def: the graph_def we want to convert

input_names: list of input names

output_names: list of output names

name: A name for the graph

opset: the opset to be used for the ONNX model, default is the latest


```
target: list of workarounds applied to help certain platforms
custom_op_handlers: dictionary of custom ops handlers
custom_rewriter: list of custom graph rewriters
extra_opset: list of extra opset's, for example the opset's used by custom ops
shape_override: dict with inputs that override the shapes given by tensorflow
inputs_as_nchw: transpose inputs in list from nhwc to nchw
outputs_as_nchw: transpose outputs in list from nhwc to nchw
large_model: use the ONNX external tensor storage format
output_path: save model to output_path
```

Returns:

An ONNX model_proto and an external_tensor_storage dict.

from_tflite

```
import tf2onnx
```



```
model_proto, external_tensor_storage = tf2onnx.convert.from_tflite(tflite_path,
    input_names=None, output_names=None, opset=None, custom_ops=None,
    custom_op_handlers=None,
    custom_rewriter=None, inputs_as_nchw=None, outputs_as_nchw=None,
    extra_opset=None,
    shape_override=None, target=None, large_model=False, output_path=None):
```

Args:

```
tflite_path: the tflite model file full path
input_names: list of input names
output_names: list of output names
opset: the opset to be used for the ONNX model, default is the latest
custom_ops: if a model contains ops not recognized by onnx runtime,
    you can tag these ops with a custom op domain so that the
    runtime can still open the model. Type is a dictionary `{op name: domain}`.
custom_op_handlers: dictionary of custom ops handlers
custom_rewriter: list of custom graph rewriters
inputs_as_nchw: transpose inputs in list from nhwc to nchw
outputs_as_nchw: transpose outputs in list from nhwc to nchw
extra_opset: list of extra opset's, for example the opset's used by custom ops
shape_override: dict with inputs that override the shapes given by tensorflow
target: list of workarounds applied to help certain platforms
large_model: use the ONNX external tensor storage format
output_path: save model to output_path
```

Returns:

An ONNX model_proto and an external_tensor_storage dict.

Creating custom op mappings from python

For complex custom ops that require graph rewrites or input / attribute rewrites using the python interface to insert a custom op will be the easiest way to accomplish the task. A dictionary of name->custom_op_handler can be passed to `tf2onnx.tfonnx.process_tf_graph`. If the op name is found in the graph the handler will have access to all internal structures and can rewrite that is needed. For example [examples/custom_op_via_python.py](#):

```
import tensorflow as tf
import tf2onnx
from onnx import helper
```



```
_TENSORFLOW_DOMAIN = "ai.onnx.converters.tensorflow"
```

```
def print_handler(ctx, node, name, args):
    # replace tf.Print() with Identity
    # T output = Print(T input, data, @list(type) U, @string message, @int first_n, @int
    summarize)
```

```

# becomes:
#   T output = Identity(T Input)
node.domain = _TENSORFLOW_DOMAIN
del node.input[1:]
return node

with tf.Session() as sess:
    x = tf.placeholder(tf.float32, [2, 3], name="input")
    x_ = tf.add(x, x)
    x_ = tf.Print(x, [x], "hello")
    _ = tf.identity(x_, name="output")
    onnx_graph = tf2onnx.tfonnx.process_tf_graph(sess.graph,
                                                custom_op_handlers={"Print":
                                                                    (print_handler, ["Identity", "mode"])}},
                                                extra_opset=
[helper.make_opsetid(_TENSORFLOW_DOMAIN, 1)],
                                                input_names=["input:0"],
                                                output_names=["output:0"])

    model_proto = onnx_graph.make_model("test")
    with open("/tmp/model.onnx", "wb") as f:
        f.write(model_proto.SerializeToString())

```

How tf2onnx works

The converter needs to take care of a few things:

1. Convert the protobuf format. Since the format is similar this step is straight forward.
2. TensorFlow types need to be mapped to their ONNX equivalent.
3. For many ops TensorFlow passes parameters like shapes as inputs where ONNX wants to see them as attributes. Since we use a frozen graph, the converter will fetch the input as constant, converts it to an attribute and remove the original input.
4. TensorFlow in many cases composes ops out of multiple simpler ops. The converter will need to identify the subgraph for such ops, slice the subgraph out and replace it with the ONNX equivalent. This can become fairly complex so we use a graph matching library for it. A good example of this is the tensorflow transpose op.
5. TensorFlow's default data format is NHWC where ONNX requires NCHW. The converter will insert transpose ops to deal with this.
6. There are some ops like relu6 that are not supported in ONNX but the converter can be composed out of other ONNX ops.
7. ONNX backends are new and their implementations are not complete yet. For some ops the converter generate ops with deal with issues in existing backends.

Step 1 - start with a frozen graph

tf2onnx starts with a frozen graph. This is because of item 3 above.

Step 2 - 1:1 conversion of the protobuf from tensorflow to onnx

tf2onnx first does a simple conversion from the TensorFlow protobuf format to the ONNX protobuf format without looking at individual ops. We do this so we can use the ONNX graph as internal representation and write helper functions around it. The code that does the conversion is in `tensorflow_to_onnx()`. `tensorflow_to_onnx()` will return the ONNX graph and a dictionary with shape information from TensorFlow. The shape information is helpful in some cases when processing individual ops. The ONNX graph is wrapped in a `Graph` object and nodes in the graph are wrapped in a `Node` object to allow easier graph manipulations on the graph. All code that deals with nodes and graphs is in `graph.py`.

Step 3 - rewrite subgraphs

In the next step we apply graph matching code on the graph to re-write subgraphs for ops like transpose and lstm. For an example looks at `rewrite_transpose()`.

Step 4 - process individual ops

In the fourth step we look at individual ops that need attention. The dictionary `_OPS_MAPPING` will map tensorflow op types to a method that is used to process the op. The simplest case is `direct_op()` where the op can be taken as is. Whenever possible we try to group ops into common processing, for example all ops that require dealing with broadcasting are mapped to `broadcast_op()`. For an op that composes the tensorflow op from multiple onnx ops, see `relu6_op()`.

Step 5 - optimize the functional ONNX graph

We then try to optimize the functional ONNX graph. For example we remove ops that are not needed, remove transposes as much as possible, de-dupe constants, fuse ops whenever possible, ...

Step 6 - final processing

Once all ops are converted and optimize, we need to do a topological sort since ONNX requires it. `process_tf_graph()` is the method that takes care of all above steps.

Extending tf2onnx

If you like to contribute and add new conversions to tf2onnx, the process is something like:

1. See if the op fits into one of the existing mappings. If so adding it to `_OPS_MAPPING` is all that is needed.
2. If the new op needs extra processing, start a new mapping function.
3. If the tensorflow op is composed of multiple ops, consider using a graph re-write. While this might be a little harder initially, it works better for complex patterns.
4. Add a unit test in `tests/test_backend.py`. The unit tests mostly create the tensorflow graph, run it and capture the output, then convert to onnx, run against a onnx backend and compare tensorflow and onnx results.
5. If there are pre-trained models that use the new op, consider adding those to `test/run_pretrained_models.py`.

License

Releases 30

 **v1.16.1** Latest
on Jan 16, 2024

[+ 29 releases](#)

Packages

No packages published

Contributors 100



[+ 86 contributors](#)

Languages

